Interface complète de FulleGenBrain_4

Spécification fonctionnelle, ergonomique, visuelle et d'intégration

Document de conception
Cohérent avec le formalisme et la théorie
compositionnelle de FulleGenBrain_4

25 octobre 2026

Abstract

Ce document spécifie une interface de travail complète pour FulleGenBrain_4. Le point d'entrée principal est un espace conversationnel continu, familier par sa structure générale aux assistants de chat modernes, mais organisé autour des objets, traces, contrats et compositions propres à FulleGenBrain_4. Des espaces de travail distincts sont prévus pour le catalogue et le cycle de vie des modèles, l'import et l'analyse de jeux de données multi-formats, la préparation et le contrôle des entraînements, les connecteurs de modèles externes dont les LLM, la recherche augmentée (RAG), l'observation, le clonage, la programmation compositionnelle, la mémoire vivante, l'ordonnancement ASCRC, les évaluations, la sécurité et l'audit.

La présente livraison est une **spécification cible d'interface**, pas la preuve qu'une application graphique intégrée est déjà livrée. Les écrans, composants, API et critères d'acceptation sont définis pour guider une implémentation réelle. Les prototypes existants peuvent alimenter certains services, mais ne constituent pas à eux seuls le runtime complet. ASCR d'eGenBrain v4 demeure inchangé. ASCRC est présenté comme un mécanisme opérationnel indépendant : il alloue des ressources et pilote l'activation réversible sans déterminer la vérité ni changer le statut épistémique d'un objet.

Contents

1 Mandat, périmètre et règles de lecture **4**
1.1 Objectif . . . 4
1.2 Statuts des capacités . . . 4
1.3 Principes non négociables . . . 4

2 Architecture d'information et navigation **5**
2.1 Structure globale de l'application . . . 5
2.2 Navigation primaire . . . 5
2.3 Niveaux de navigation et repères . . . 6

3 Design visuel, ergonomie et accessibilité **6**
3.1 Direction visuelle . . . 6
3.2 Accessibilité et opérations clavier . . . 7
3.3 Ergonomie des formulaires longs . . . 7

4 Espace Chat : interface centrale **8**
4.1 Structure du chat . . . 8
4.2 Contrat comportemental du chat . . . 8
4.3 Modes d'interaction . . . 8
4.4 Commandes et objets issus d'une conversation . . . 9

5 Gestion complète des modèles **9**
5.1 Catalogue de modèles . . . 9
5.2 Chargement de nouveaux modèles et formats . . . 10
5.3 Vue détaillée d'un modèle . . . 10

6 Datasets : import, schémas et champs clés **10**
6.1 Parcours d'import guidé . . . 10
6.2 Mapping des champs . . . 10
6.3 Contrôles de qualité, vie privée et licence . . . 11
6.4 Partitions et prévention de fuite . . . 11

7 Entraînement et expérimentation **11**
7.1 Assistant de configuration . . . 11
7.2 Paramètres et compatibilité . . . 12
7.3 Suivi en direct . . . 12
7.4 Promotion de modèle . . . 12

8 Intégration optionnelle de LLM, eLLM, modèles tiers et RAG **12**
8.1 Principe d'isolation . . . 12
8.2 Fiche de connecteur LLM . . . 12
8.3 Espace RAG . . . 13
8.4 Évaluation et garde-fous . . . 13

9 Observation : données, trajectoires et instruments **13**
9.1 Écran de l'observatoire . . . 13
9.2 Modèle d'une observation . . . 13
9.3 Comparaison multi-vues et collecte active . . . 13

10 Clonage et reproduction **14**
10.1 Objet du clonage . . . 14
10.2 Parcours de clonage . . . 14
10.3 Rapport de fidélité . . . 14

11 Programmation compositionnelle : IDE intégré **14**
11.1 Vue de travail . . . 14
11.2 Panneau de composant . . . 15
11.3 Vérification et exécution . . . 15
11.4 Couverture des opérateurs . . . 15
11.5 Diff, révision et publication . . . 15

12 Mémoire vivante et gestion des ressources **15**
12.1 Explorateur de mémoire . . . 15
12.2 Vagues d'information et propagation . . . 16

13 ASCRC : pilotage opérationnel **16**
13.1 Périmètre et séparation de l'ASCR existant . . . 16
13.2 Vue ASCRC . . . 16
13.3 Politique d'allocation . . . 16
13.4 Veille, réveil, protection et équité . . . 16

14 Évaluations, modèles du monde et décisions de promotion **17**
14.1 Centre d'expérimentation . . . 17
14.2 Évaluation d'un modèle du monde . . . 17
14.3 Rapport de campagne . . . 17

15 Objectifs, tâches et travail autonome contrôlé **17**

16 Sécurité, permissions et isolation **17**
16.1 Règles de sécurité d'interface 17
16.2 États et erreurs 18

17 Traçabilité, audit et provenance **18**

18 Paramètres, préférences et administration **18**

19 Modèle d'objets d'interface et contrat d'API **18**
19.1 Objets canoniques 18
19.2 Contrat des services 19
19.3 Couche d'intégration et adaptateurs 19

20 États transversaux et feedback utilisateur **19**

21 Critères d'acceptation de l'interface **19**

22 Plan d'implémentation conseillé **20**

23 Risques d'ergonomie et mesures préventives **21**

24 Conclusion et statut de la livraison **22**

A Matrice de couverture fonctionnelle de l'interface **22**

B Règle finale de non-confusion **23**

1 Mandat, périmètre et règles de lecture

1.1 Objectif

L'interface doit rendre FulleGenBrain_4 utilisable sans réduire l'architecture à une fenêtre de conversation. Elle doit relier le dialogue aux objets durables : tâches, observations, données, modèles, compositions, exécutions, preuves, expériences, versions, dépendances et événements mémoire. L'utilisateur doit pouvoir partir d'une question, poursuivre dans le chat, inspecter ce que le système a réellement fait, composer ou corriger un mécanisme, évaluer une variante, puis sauvegarder ou promouvoir la version choisie.

1.2 Statuts des capacités

Les marqueurs ci-dessous doivent apparaître dans ce document, dans les prototypes d'écran et dans le futur produit quand une distinction est importante.

Marqueur	Sens
[CIBLE]	Comportement exigé par la présente spécification.
[PROTOTYPE LOCAL]	Service ou comportement démontré dans un prototype isolé et borné.
[ADAPTATEUR]	Capacité dépendant d'un connecteur, d'un pilote, d'un parseur ou d'un fournisseur explicitement configuré.
[À INTÉGRER]	Fonction dont l'interface est spécifiée, mais dont le service complet doit encore être réalisé et raccordé.
[À MESURER]	Fonction dont l'utilité ou les performances ne peuvent être déclarées sans protocole et résultats empiriques.

Ces marqueurs ne classent pas une idée comme impossible ou inutile. Ils empêchent simplement de confondre un écran prévu, un contrat défini, un test local et une intégration validée.

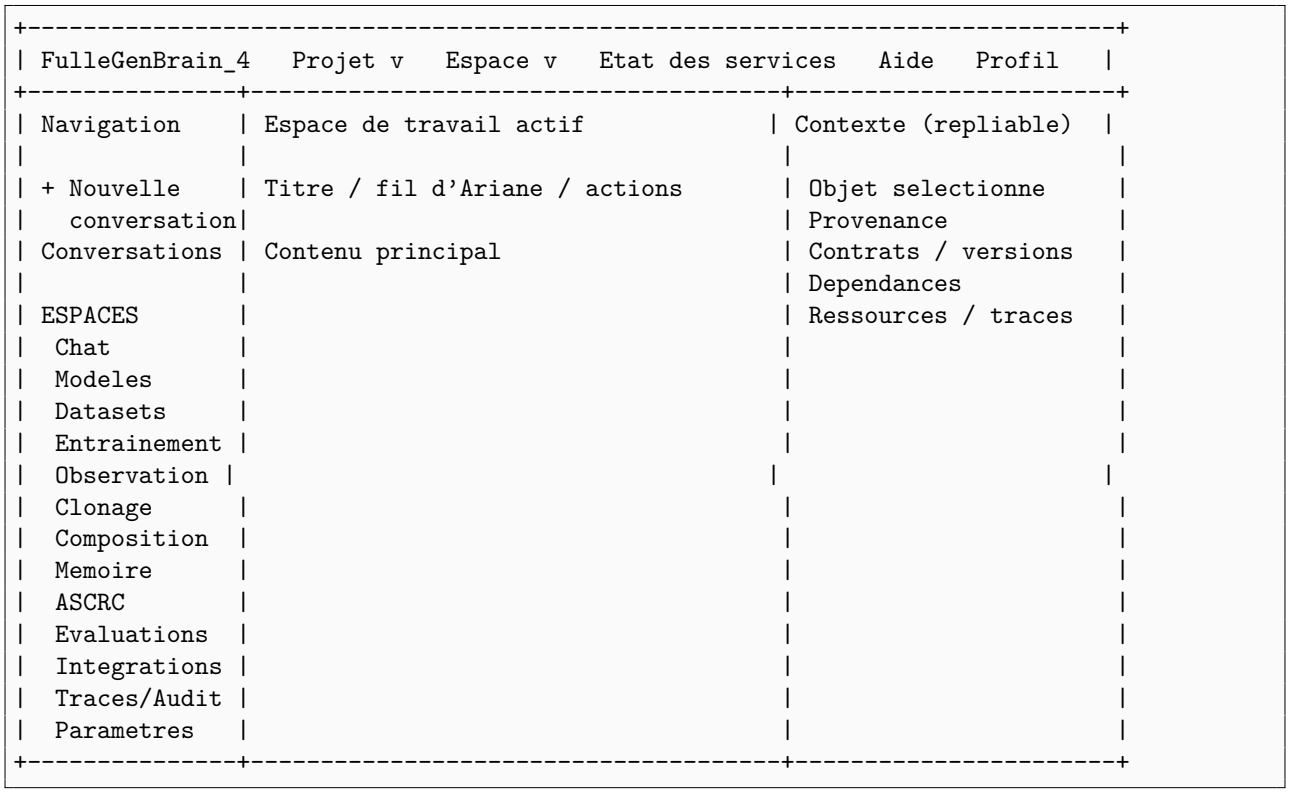
1.3 Principes non négociables

1. **La composition est un parcours de premier ordre.** Les composants, leurs types, effets, contrats, preuves, dépendances et versions sont consultables et manipulables depuis l'interface.
2. **Le chat est une porte d'entrée, pas une boîte noire.** Une réponse importante peut exposer sa provenance, ses étapes calculées, les objets utilisés, les vérifications exécutées et les limites reconnues.
3. **Un statut épistémique n'est jamais un état de ressources.** « Dormant », « en attente » ou « chargé » ne signifie ni vrai, ni faux, ni validé, ni rejeté.
4. **ASCR reste intact.** L'interface ne réécrit pas l'ASCR d'eGenBrain v4. Les commandes de contrôle opérationnel relèvent d'ASCRC, dans un espace séparé et auditable.
5. **Pas de dépendance LLM implicite.** Le cœur reste utilisable sans LLM externe. Tout LLM, modèle importé ou service RAG est optionnel, visible, désactivable et gouverné par des permissions.
6. **Pas de réussite fictive.** Une tâche incomplète, une validation non effectuée, un import partiel, une connexion perdue ou une preuve manquante sont affichés comme tels.
7. **Réversibilité et provenance.** Toute activation, promotion, transformation mémoire, clonage, entraînement ou modification compositionnelle produit un événement versionné et permet de retrouver sa source et ses conséquences.

2 Architecture d'information et navigation

2.1 Structure globale de l'application

L'application est organisée en trois zones adaptatives : une barre latérale de navigation et de projets; un espace de travail principal; un panneau contextuel repliable qui montre les détails pertinents à la sélection courante. En grand écran, la navigation demeure persistante, le panneau contextuel peut être ouvert à droite, et l'espace de travail conserve la priorité visuelle. Sur écran étroit, navigation et panneau deviennent des tiroirs superposés sans réduire le contenu à une grille illisible.



2.2 Navigation primaire

Espace	But principal	Objets et actions clés
Chat	Dialogue et orchestration de tâches	Conversations, pièces jointes, objectifs, réponses, plans d'action, citations/provenance, tâches longues et résultats.
Modèles	Inventaire et cycle de vie des modèles	Import, inspection, métadonnées, versions, chargement/déchargement, adaptateurs, promotion et retour arrière.
Datasets	Import, préparation et gouvernance des données	Fichiers, connecteurs, schémas, champs clés, qualité, déduplication, transformations, licences, divisions train/validation/test.
Entraînement	Configurer, lancer, reprendre et comparer	Configurations versionnées, paramètres, ressources, journaux, métriques, checkpoints, critères d'arrêt.

Observation	Explorer les observations et trajectoires	Sources, sessions, environnement, événements, vues synchronisées, incertitudes et instruments.
Clonage	Reproduire un comportement ou mécanisme observé	Périmètre de clone, sources, hypothèses, stratégie de reproduction, tests de fidélité, écarts et provenance.
Composition	Programmer par composants composables	Catalogue, éditeur de graphe/AST, types, contrats, effets, exécution de test, diffs et versions.
Mémoire vivante	Gérer continuité et cycle de vie mémoire	Objets actifs/dormants/périmés/archivés/révoqués, vagues, checkpoints, dépendances et restauration.
ASCRC	Allouer ressources et gérer activation opérationnelle	Budget dur, files, candidats, quotas, veille/réveil, politiques, blocages et journal d'arbitrage.
Évaluations	Mesurer le comportement réel	Scénarios, suites de test, baselines, ablations, métriques, rapports et seuils de promotion.
Intégrations	Connecter services externes sans dépendance implicite	LLM, RAG, stockage, embeddings, outils, systèmes de données, secrets, permissions et tests de connexion.
Traces / Audit	Reconstruire une exécution	Événements, graphes de dépendances, budgets, décisions, erreurs, versions et export signé.
Paramètres	Contrôler l'espace de travail	Identité, thème, langue, raccourcis, politiques, sécurité, stockage et administration.

2.3 Niveaux de navigation et repères

Chaque écran présente un titre explicite, un fil d'Ariane court, la portée active (projet / espace de travail / session) et l'état de sauvegarde. Une recherche globale porte sur les noms, identifiants, contenus textuels indexés, tags, conversations, modèles, datasets, compositions et événements, selon les permissions. Les résultats sont filtrables par type, version, état, provenance et période. Aucun résultat ne contourne les droits d'accès.

Le menu « + » permet de créer le type d'objet cohérent avec l'espace courant; il ne remplace pas les actions contextuelles. Les actions destructives, les changements de politique de sécurité, les suppressions irréversibles et les promotions de modèle nécessitent un résumé d'impact et une confirmation explicite. Les opérations réversibles possèdent une action « Annuler » ou « Restaurer » lorsqu'elle est techniquement possible.

3 Design visuel, ergonomie et accessibilité

3.1 Direction visuelle

La direction proposée est sobre, technique mais non froide : fond neutre, cartes peu encadrées, typographie lisible, espaces réguliers, accent chromatique unique pour les actions principales, couleurs de statut limitées et cohérentes. L'interface doit paraître comme un outil de travail continu, pas comme un tableau de bord rempli de widgets. La couleur seule ne doit jamais porter une information : les statuts utilisent un libellé, une icône et éventuellement une couleur.

Token de design	Règle cible
Grille	Basée sur une unité de 4 px; espacements usuels 8, 12, 16, 24 et 32 px.
Typographie	Police UI sans empattement système; texte courant de 14–16 px; titres hiérarchisés et courts; code en police monospace.
Largeur de lecture	Texte narratif autour de 65–85 caractères par ligne; tables techniques défilables ou reconfigurables sur petit écran.
Surfaces	Fond principal neutre, arrière-plan légèrement différencié pour navigation et panneaux; cartes réservées aux groupes réellement indépendants.
Action primaire	Une action principale évidente par écran ou formulaire; les actions secondaires restent visuellement discrètes.
Densité	Vues compactes pour tables expertes, vues détaillées à la demande, préférences de densité persistantes.
Couleurs de statut	Information portée par libellé et icône; couleurs de danger réservées aux risques ou actions destructives.
Mouvement	Transitions rapides et modestes; respect de *prefers-reduced-motion*; aucune information uniquement animée.
Thèmes	Clair et sombre; contraste vérifié dans les deux; préférence système prise en charge.

3.2 Accessibilité et opérations clavier

- Toutes les actions sont utilisables au clavier; le focus est visible et suit un ordre logique. Les modales piègent le focus tant qu'elles sont ouvertes et le restituent à leur déclencheur à la fermeture.
- Les composants sémantiques exposent leur rôle, nom, état, description et erreurs aux technologies d'assistance. Les changements d'état non intrusifs utilisent des annonces accessibles.
- Cible normative : WCAG 2.2 niveau AA pour les parcours pris en charge, à vérifier par audit automatique et manuel. L'interface ne prétend pas satisfaire ce niveau avant l'audit.
- Les champs affichent leur libellé en permanence; les placeholders ne servent jamais de seul libellé. Les erreurs sont proches du champ et expliquent comment les résoudre.
- Les graphes d'observation et de composition offrent des équivalents tabulaires/listés et une navigation sans souris. Les couleurs de graphe sont doublées par des symboles et libellés.
- Les raccourcis sont découvrables, configurables sans collision avec les technologies d'assistance et désactivables. Les raccourcis globaux essentiels : nouvelle conversation, recherche, navigation, ouvrir l'aide et aller à la zone de saisie.

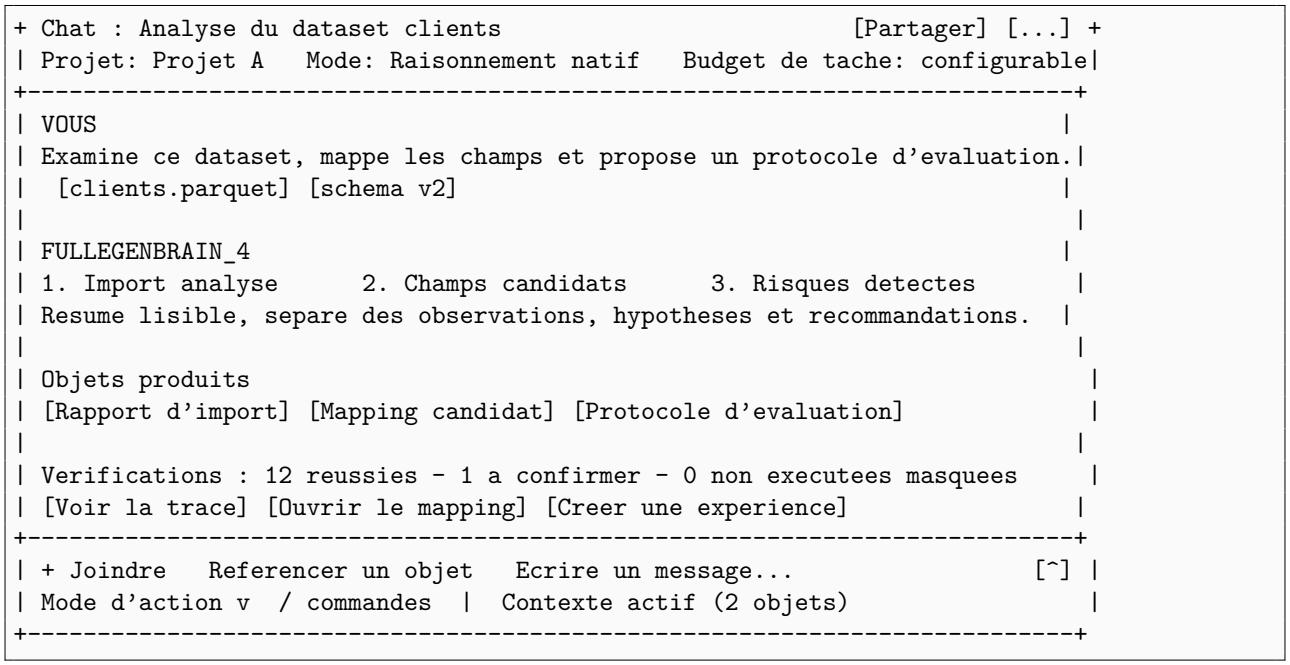
3.3 Ergonomie des formulaires longs

Les formulaires d'entraînement, d'import et de connexion sont divisés en étapes révisables. Ils préservent les saisies en brouillon, permettent de revenir en arrière et affichent une synthèse avant toute opération coûteuse. Les paramètres avancés sont repliés par défaut mais restent faciles à trouver; les valeurs modifiées par rapport au profil de base sont marquées. Une validation côté client améliore la fluidité, mais la validation côté service demeure autoritaire.

4 Espace Chat : interface centrale

4.1 Structure du chat

Le chat est une conversation à défilement stable, avec un en-tête léger, les messages et une zone de saisie persistante en bas. La zone de saisie accepte du texte, des fichiers, des références à des objets, des commandes d'espace de travail et des actions structurées. Le contexte utilisé est visible sous forme de pièces jointes ou de références sélectionnées, jamais comme un contexte implicite impossible à inspecter.



4.2 Contrat comportemental du chat

1. Chaque message possède un identifiant stable, une date, un auteur ou service, une portée de conversation et, le cas échéant, des pièces jointes, sources, objets créés et traces d'exécution.
2. Les réponses qui décrivent des opérations réelles distinguent : résultat obtenu, étape non exécutée, proposition, inférence, incertitude, erreur et prochaine action possible.
3. Les actions qui changent l'état du système sont présentées comme commandes explicites (« importer », « démarrer l'entraînement », « activer », « révoquer ») avec portée, ressources et conséquences. Le texte conversationnel seul ne contourne pas l'autorisation requise.
4. L'utilisateur peut arrêter une opération, examiner son état, reprendre une opération reprenable, ou annuler une modification transactionnelle lorsque le contrat le permet.
5. Les réponses volumineuses peuvent être repliées en sections (résumé, détails, objets, preuves, logs) sans cacher par défaut un avertissement critique, une erreur ou une limite.
6. Les messages et résultats sont partageables dans le périmètre autorisé, exportables et référençables depuis les espaces dédiés. Un lien d'objet ouvre sa fiche versionnée plutôt qu'une copie désynchronisée.

4.3 Modes d'interaction

Le chat doit proposer des modes qui modifient la forme du travail, non le statut de vérité :

- **Dialogue :** discussion et exploration, sans exécution implicite d'action destructive.
- **Tâche structurée :** l'objectif, les entrées, les sorties attendues, les contraintes et le budget sont explicités.

- **Programmation compositionnelle :** la conversation peut générer ou éditer un AST/graph de composition candidat, soumis ensuite au typage, aux contrats et aux tests.
- **Observation :** sélection de sources et d'instruments, avec collecte des observations et provenance.
- **Audit :** explication focalisée sur la trace, les hypothèses, les dépendances et les raisons de décision.

Un mode LLM externe n'est accessible qu'après activation d'un connecteur autorisé et reste clairement étiqueté avec le fournisseur, le point de terminaison, les modalités de traitement des données et les limites connues.

4.4 Commandes et objets issus d'une conversation

Les commandes reconnues ouvrent une prévisualisation structurée avant exécution. Les objets créés reçoivent un identifiant, une version, une provenance et un statut. La conversation ne devient pas le magasin canonique du modèle ou du dataset : elle est une vue de travail reliée aux référentiels dédiés. Les pièces jointes ne sont pas automatiquement traitées comme fiables ou autorisées pour tous les usages.

5 Gestion complète des modèles

5.1 Catalogue de modèles

Le catalogue réunit les modèles natifs, modèles du monde, modèles spécialisés, modèles importés et adaptateurs externes. Les types de modèle sont indiqués séparément de leur état opérationnel. La fiche modèle expose au minimum : identifiant, nom, famille, version, format, taille, empreinte, provenance, licence, propriétaire, environnement, ressources requises, entrées/sorties, dépendances, métriques, jeux de données d'entraînement autorisés, statut de validation, approbation, endpoint éventuel et historique des changements.

Commande	Règles d'interface et de service
Importer / enregistrer	Choisir fichier, répertoire, registre ou adaptateur; inspecter le manifeste, l'empreinte, les formats et la licence; quarantaine avant chargement exécutable.
Inspecter	Afficher métadonnées, signatures, architecture déclarée, dépendances, espace disque et compatibilités connues; ne pas inférer une architecture non inspectable.
Charger	Prévalider la mémoire et les ressources; afficher appareil cible, coût estimé, durée si mesurée, provenance et plan de déchargement; laisser le service décider de la faisabilité réelle.
Décharger / mettre en veille	Distinguer du retrait du catalogue; vérifier références actives, leases, transactions et tâches en cours; offrir une alternative si la protection bloque l'opération.
Comparer	Comparer des versions sur leurs métadonnées, configurations, tests, métriques et coût, sans supposer des métriques comparables entre tâches différentes.
Promouvoir / restaurer	Exiger les critères d'acceptation définis, un rapport d'évaluation, une version cible et un plan de retour arrière.
Révoquer	Enregistrer motif, périmètre, dépendances affectées, date et acteur; ne pas effacer l'historique d'audit.

5.2 Chargement de nouveaux modèles et formats

Le parcours d'import comporte : sélection de provenance; inspection non exécutante; validation du format et de l'empreinte; lecture de licence et de consentement; scan de sécurité; choix de l'adaptateur; test de compatibilité; configuration des ressources; essai isolé; inscription au catalogue. Les formats supportés sont déclarés par adaptateur au moment de l'installation. L'interface doit montrer la liste réellement prise en charge par l'environnement et ne doit pas promettre qu'un simple nom de fichier suffit.

Les familles à prévoir dans l'architecture d'adaptateurs incluent les formats sérialisés natifs, les graphes/poids d'apprentissage, les modèles tabulaires, les modèles de séries temporelles, les modèles multimodaux et les endpoints de modèles servis à distance. Exemples de formats possibles selon adaptateur : ONNX, safetensors, formats de frameworks d'apprentissage courants, bundles de modèle versionnés et API compatibles. L'affichage doit énumérer les compatibilités testées, la version du runtime nécessaire et les fonctions indisponibles. Tout chargement dans un processus séparé doit être isolé, avec permissions minimales et ressources limitées.

5.3 Vue détaillée d'un modèle

La fiche comprend des onglets *Résumé*, *Versions*, *Contrat I/O*, *Datasets et provenance*, *Évaluations*, *Ressources*, *Dépendances*, *Exécutions* et *Audit*. L'action « utiliser dans une composition » ouvre un sélecteur typé qui vérifie la compatibilité des entrées, sorties, effets et permissions. Le modèle ne devient pas automatiquement une compétence approuvée parce qu'il est importé ou chargé.

6 Datasets : import, schémas et champs clés

6.1 Parcours d'import guidé

Le gestionnaire de datasets doit accepter des fichiers locaux, des répertoires et des connecteurs explicitement configurés. Le système annonce les formats effectivement supportés par chaque lecteur. La cible produit une architecture extensible pour les données tabulaires (CSV, TSV, tableurs, Parquet/Arrow), structurées (JSON, JSONL, XML selon lecteur), textuelles (TXT, Markdown), documents (PDF selon extraction), ainsi que des médias (image, audio, vidéo) si les analyseurs requis sont installés. Le support d'un format ne garantit ni la qualité de l'extraction ni la compréhension sémantique du contenu.

Étapes : (1) sélection et provenance; (2) aperçu contrôlé; (3) détection du format et de l'encodage; (4) choix du parseur; (5) schéma et champs candidats; (6) définition du rôle des colonnes; (7) qualité et déduplication; (8) gestion des données sensibles; (9) divisions expérimentales; (10) validation et versionnement. L'assistant d'import peut proposer des choix, mais l'utilisateur peut les inspecter et les corriger avant publication.

6.2 Mapping des champs

Pour chaque dataset, fournir une table éditable avec : nom source, nom canonique, type inféré, type confirmé, rôle, statut nullable, valeurs manquantes, cardinalité, exemples masqués, règles de validation, transformation, unité, langue, sensibilité, clé primaire candidate, rôle cible et justification. Les types incluent scalaire, date/heure avec fuseau, catégorie, texte, tableau, objet imbriqué, image/audio/vidéo ou opaque non interprété. Une donnée dont le type est incertain est conservée comme opaque et signale l'incertitude.

Champ d'interface	Contrat
Clé d'enregistrement	Identifier la clé choisie, la source de son unicité, les collisions et la stabilité entre versions. Les clés synthétiques doivent être distinguées des identifiants natifs.

Champ cible / label	Définir l'objet à prédire ou mesurer; avertir si une cible est absente, ambigüe, dérivée d'une fuite d'information ou indisponible au moment de la décision.
Champs de groupe	Permettre des divisions par personne, session, appareil, patient, document, source ou autre groupe pour limiter la fuite entre partitions.
Temps et ordre	Signaler fuseau, granularité, doublons temporels et possibilités de fuite du futur; offrir un découpage temporel lorsque pertinent.
Features / attributs	Distinguer les champs utilisables, interdits, sensibles, de provenance, de contexte ou réservés à l'audit.
Poids / masque / métadonnées	Supporter des rôles supplémentaires si l'algorithme les exige; faire valider les noms et types par l'adaptateur du modèle.

6.3 Contrôles de qualité, vie privée et licence

Les diagnostics incluent enregistrement vide, colonnes constantes, valeurs manquantes, doublons exacts et quasi-doublons, conflits de types, distributions atypiques, valeurs hors domaine, classes rares, déséquilibres, fuite potentielle de données, contamination entre partitions, erreurs d'encodage et anomalies de temporalité. Ces diagnostics sont des alertes, non des corrections automatiques cachées. Toute transformation conserve sa configuration, sa portée, le nombre d'enregistrements affectés et un échantillon de contrôle conforme aux règles de confidentialité.

La fiche dataset conserve la source, le droit d'utilisation, le consentement si applicable, la rétention, les conditions de partage, le propriétaire, la date d'import, l'empreinte, la méthode d'extraction et les transformations. Les secrets, identifiants directs et champs sensibles sont masqués dans les aperçus par défaut. Les exports et le partage requièrent une permission adaptée.

6.4 Partitions et prévention de fuite

Le parcours propose, selon la tâche, une partition aléatoire reproductible, stratifiée, groupée, temporelle, par source ou définie manuellement. L'interface affiche la proportion et les effectifs, les entités recouvrantes, les dates de coupure, le seed et les transformations apprises uniquement sur les partitions autorisées. Les partitions test sont verrouillées par défaut pour le réglage du modèle. Une partition personnalisée exige un aperçu et des avertissements, pas une acceptation silencieuse.

7 Entraînement et expérimentation

7.1 Assistant de configuration

L'entraînement est un workflow versionné dont les étapes sont :

1. **But :** tâche, unité d'évaluation, cible, domaine de validité et critères d'arrêt.
2. **Données :** dataset et version, mapping, transformations, partitions, filtres et provenance.
3. **Modèle :** modèle de départ, stratégie d'initialisation, tête/tâche ou adaptateur requis, compatibilité des types.
4. **Méthode :** algorithme, fonction objectif, optimiseur, calendrier, régularisation, critères d'arrêt et méthode de sélection.
5. **Paramètres :** seed, epochs/itérations, batch size, taux d'apprentissage, accumulation, précision, taille de contexte ou paramètres propres à l'algorithme.
6. **Ressources :** CPU/GPU/accélérateur, mémoire, nombre de workers, stockage, quotas énergétiques si mesurés, priorité et plafond de coût.
7. **Validation :** métriques, seuils, baseline, ablations, jeux tenus à l'écart, tests de sécurité et répétitions.

8. **Résumé :** diff de configuration, coûts estimés avec incertitude, risques, éléments non configurés et confirmation de lancement.

7.2 Paramètres et compatibilité

Les champs de paramètres sont fournis par le type de tâche et l'adaptateur du modèle. L'interface ne doit pas présenter un paramètre générique si le backend ne le comprend pas. Les paramètres par défaut sont documentés et versionnés. Chaque paramètre indique valeur effective, unité, portée, provenance (défaut ou saisie) et validation. Les valeurs hors plage ou incompatibles bloquent le démarrage avec explication actionnable.

L'utilisateur peut sauvegarder une configuration avant lancement, dupliquer une expérience et comparer les changements. Les configurations sont immuables après démarrage; les changements produisent une nouvelle révision et un nouveau run. Toute reprise de checkpoint vérifie les versions du code, de l'adaptateur, du dataset, des dépendances et la compatibilité du checkpoint.

7.3 Suivi en direct

L'écran d'exécution affiche au moins : état de tâche; étape courante; temps écoulé; compteur d'itérations/époques; progression si connue; métriques train et validation; courbes avec données accessibles sous forme de tableau; ressources CPU/GPU/mémoire; débit; erreurs et avertissements; checkpoint disponible; version de configuration; journaux filtrables. Le temps restant n'est affiché que si une estimation raisonnable est disponible et doit être indiquée comme estimation.

Actions : pause si prise en charge, arrêt propre, annulation avec confirmation, reprendre depuis checkpoint, ouvrir la trace, copier la configuration, comparer, exporter les journaux et créer une nouvelle expérience depuis celle-ci. Le bouton d'arrêt ne promet pas un état terminé avant l'accusé du service. Les runs échoués conservent leurs diagnostics et données de provenance; ils ne sont pas supprimés automatiquement.

7.4 Promotion de modèle

La promotion est une décision distincte de la fin de l'entraînement. La page résume les résultats hors entraînement, la comparaison à la baseline, les incertitudes statistiques, les cas d'échec, les coûts, les tests de régression, les critères non satisfaits et les effets sur les compositions dépendantes. Une métrique isolée ou une perte d'entraînement faible ne suffit pas à marquer le modèle comme validé. La promotion nécessite une politique explicite; le retour arrière préserve la version promue précédente.

8 Intégration optionnelle de LLM, eLLM, modèles tiers et RAG

8.1 Principe d'isolation

Le support d'un LLM externe ou d'un RAG est un **connecteur optionnel**, pas une dépendance du noyau FulleGenBrain_4. Un administrateur peut désactiver tous les connecteurs et conserver les fonctions natives disponibles. L'interface étiquette clairement les réponses dont la génération ou la récupération dépend d'un service tiers. Aucun texte provenant d'un modèle, d'un document ou d'un outil externe n'est élevé automatiquement au rang de vérité, de preuve ou de règle certifiée.

8.2 Fiche de connecteur LLM

La configuration contient : nom et type du fournisseur; endpoint local ou distant; méthode d'authentification par gestionnaire de secrets; modèle et version; paramètres disponibles; limites de débit/contexte; résidence et traitement des données déclarés; politique de journalisation; délais et reprises; permissions d'outils; plafonds de coûts; tests de santé; état de connexion; dernière vérification. Les secrets ne sont jamais affichés en clair ni enregistrés dans les journaux. Les erreurs distinguent authentification, indisponibilité, quota, délai dépassé, réponse invalide et refus du fournisseur.

8.3 Espace RAG

Le RAG possède un espace dédié pour les sources, collections, connecteurs, parseurs, transformations, segmentations, index, modèles d'embedding ou alternative déclarée, stratégies de recherche, filtres, reranking, seuils, contexte envoyé et résultats d'évaluation. Chaque pipeline est versionné. La fiche d'une réponse RAG expose les extraits exacts récupérés, leur provenance, leur version, leur position, le score de récupération lorsqu'il est disponible, les filtres appliqués et les documents ignorés. Le score de similarité ne vaut pas preuve de vérité.

Le parcours de constitution d'un index présente les comptes de documents découverts, réussis, ignorés et échoués; la cause des échecs; l'empreinte des sources; la politique de mise à jour/suppression; et un échantillon de contrôle. Lorsqu'une source est modifiée ou retirée, les segments dérivés sont invalidés/recalculés selon le contrat du connecteur. Les documents conflictuels restent comparables plutôt que fusionnés silencieusement.

8.4 Évaluation et garde-fous

Un connecteur LLM/RAG doit être évalué sur des jeux de requêtes représentatifs, avec critères de pertinence, fidélité aux sources, refus, fuite de données, robustesse, latence et coût. Comparaisons recommandées : sans connecteur, modèle nul/permuté lorsque pertinent, et alternatives à budget comparable. La promotion est limitée à un contexte et à une version de configuration. Le bouton « déployer » reste distinct de « connecter » et de « tester ».

9 Observation : données, trajectoires et instruments

9.1 Écran de l'observatoire

L'observatoire sert à collecter et comprendre ce qui a été effectivement observé, sans confondre observation et interprétation. Il dispose de vues synchronisées selon la modalité : timeline des événements, table structurée, détails d'un objet, aperçu média, graphe de dépendance et panneau provenance. Le type de vue dépend des analyseurs disponibles et de leurs capacités déclarées.

L'en-tête choisit une source, session, période, environnement, point d'observation et budget. La timeline montre les événements horodatés, le fuseau, la source, la version du capteur/adaptateur, la qualité, les pertes de collecte et les événements non synchronisés. Un événement est consultable dans son format original lorsqu'il est permis, ainsi que dans ses représentations dérivées.

9.2 Modèle d'une observation

Chaque observation expose : ID et version; source; moment de collecte et incertitude temporelle; contexte; capteur/adaptateur et configuration; payload ou référence protégée; transformation; confiance technique si calibrée; éléments manquants; permissions; empreinte et liens vers les dérivés. Les interprétations possibles sont enregistrées comme hypothèses séparées avec conditions de falsification et données discriminantes souhaitées.

9.3 Comparaison multi-vues et collecte active

L'utilisateur peut synchroniser plusieurs flux, comparer des fenêtres, filtrer, annoter, relier un événement à une composition ou une hypothèse, puis demander une nouvelle collecte si un instrument disponible le permet. Avant une collecte active, l'interface montre les permissions, effets, coût, risque et condition d'arrêt. Les informations non recueillies ne sont pas dessinées comme si elles existaient. Les transformations irréversibles ou destructrices ne sont pas offertes comme simples filtres de visualisation.

10 Clonage et reproduction

10.1 Objet du clonage

Le clonage est traité comme la reproduction explicite d'une portée observable, d'un comportement, d'une procédure, d'un environnement ou d'un artefact autorisé. Il n'implique pas à lui seul la reconstruction exacte de mécanismes internes ou d'états non observables. L'écran impose de choisir ce qui est cloné et comment la fidélité sera mesurée.

10.2 Parcours de clonage

1. Choisir la cible et la portée : comportement, interface, procédure, modèle, composition, environnement de test ou sous-système.
2. Sélectionner les sources admissibles : observations, traces, fichiers, spécifications et versions; vérifier droits et provenance.
3. Déclarer les propriétés à préserver : entrées/sorties, invariants, timing, séquences d'événements, ressources, règles de sécurité, distribution ou contrat de comportement.
4. Enregistrer les hypothèses, les dimensions non observées et les degrés de liberté.
5. Choisir la stratégie de reproduction : duplication fidèle d'un artefact, composition de composants, synthèse de programme, identification de paramètres ou approximation comportementale.
6. Générer un candidat isolé; produire sa carte de provenance et la liste des éléments inférés versus copiés.
7. Construire un protocole d'écarts : cas de référence, cas limites, perturbations, scénarios tenus à l'écart et critères d'échec.
8. Tester, comparer, inspecter les écarts et itérer. Une promotion nécessite les critères préalablement définis.

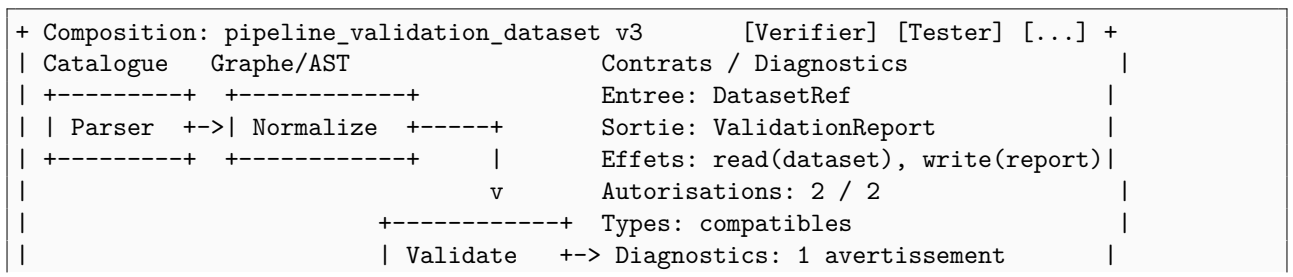
10.3 Rapport de fidélité

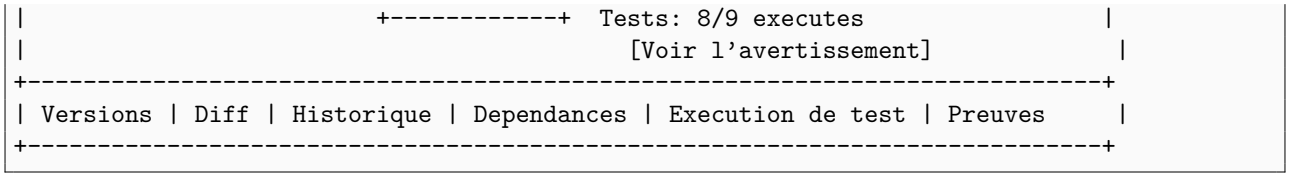
Le rapport distingue identité byte-à-byte, équivalence structurelle, équivalence des contrats, équivalence sur les cas observés et similarité statistique. Ces catégories ne sont pas interchangeables. Il affiche les cas passés/échoués/non testés, les conditions de l'essai, les sources des exemples, les limites de généralisation et les hypothèses non identifiées. Les candidats clonés ne remplacent pas la cible et n'héritent pas automatiquement de ses statuts épistémiques.

11 Programmation compositionnelle : IDE intégré

11.1 Vue de travail

L'éditeur compositionnel propose deux représentations synchronisées : vue graphique du graphe/AST et vue textuelle sérialisée. Les utilisateurs peuvent explorer le catalogue, déposer des composants, connecter des ports, éditer des expressions, définir des branches, des reprises et des transactions, puis inspecter les types, effets, contrats et dépendances. La vue textuelle reste une représentation source contrôlable, non un simple export opaque de la vue graphique.





11.2 Panneau de composant

Chaque composant expose identité/version, signature, type, pré/postconditions, effets, permissions, ressources, exceptions, tests, preuves ou obligations, provenance, dépendances, statut de validation et compatibilité. Les composants non implémentés peuvent être affichés comme concepts théoriques ou adaptateurs absents, mais ils sont explicitement non exécutables. L'éditeur ne doit pas permettre de publier un graphe qui contient des nœuds fictifs sous l'apparence de nœuds fonctionnels.

11.3 Vérification et exécution

Le pipeline visible dans l'interface suit les étapes disponibles : syntaxe; résolution des références; typage; vérification des effets/capacités; vérification des contrats; analyse de dépendances et cycles; planification des ressources; exécution en bac à sable; tests; revue des traces; enregistrement du résultat. L'absence d'un vérificateur est un état « non vérifié » ou « non disponible », pas une réussite.

La fiche d'exécution présente les entrées, versions, sortie, effets observés, coût, environnement, traces, erreurs et résultats de contrôle. Les tests de propriétés, cas limites, régression, contre-exemples et tests différentiels sont des objets versionnés. Un résultat réussi dans le bac à sable ne valide pas automatiquement le déploiement dans un autre contexte.

11.4 Couverture des opérateurs

La théorie comporte un espace d'opérateurs plus vaste que le prototype intégré. Le catalogue doit donc disposer d'une matrice de disponibilité par opérateur : défini syntaxiquement; typé; sémantique opérationnelle; contrats et effets; tests; preuve ou obligations ouvertes; implémentation et version du runtime. Les opérateurs non intégrés restent visibles dans le catalogue théorique, avec capacités désactivées et chemin de réalisation. La couverture ne doit être ni cachée ni artificiellement ramenée au plus petit dénominateur.

11.5 Diff, révision et publication

Les diffs couvrent le graphe, la sérialisation, les signatures, les contrats, les permissions, les dépendances et les paramètres d'exécution. Toute publication créé une version immuable, rattache tests et trace, et indique les consommateurs affectés. Un changement incompatible provoque un diagnostic et requiert une migration ou un verrouillage de version. Le retour arrière restaure une version antérieure sans supprimer les traces de la version révoquée.

12 Mémoire vivante et gestion des ressources

12.1 Explorateur de mémoire

La mémoire est consultable selon type, état opérationnel, statut épistémique, version, fraîcheur, provenance, coût, dépendances et motif de conservation. Les catégories d'interface doivent distinguer : actif, dormant, périmé (stale), bloqué, révoqué, archivé et absent du catalogue. Les états disponibles varient par type d'objet; leur mapping est déclaré par adaptateur.

Les opérations de mémoire comprennent : inspecter; ouvrir/réveiller; mettre en veille; créer un checkpoint; restaurer; invalider; réconcilier une version; compresser un payload; résumer avec une transformation tracée; archiver; retirer du service; révoquer. « Dormir » ne signifie pas « supprimer

» et la compression n'est pas implicitement sans perte. Toute transformation indique propriétés préservées, pertes possibles, coût, hash, source, version et méthode de restauration.

12.2 Vagues d'information et propagation

L'écran des vagues affiche source, destinataire, type d'événement, version source, contexte, séquence, priorité, durée de vie, provenance, état de livraison, duplication et erreurs. Une vague doit respecter les autorisations, la version et les limites de débit. Les événements de révocation et de changement de version doivent invalider ou réévaluer les descendants pertinents avant de traiter les demandes exploratoires de moindre priorité. L'interface propose une simulation du périmètre affecté avant une propagation en production.

13 ASCRC : pilotage opérationnel

13.1 Périmètre et séparation de l'ASCR existant

ASCR d'eGenBrain v4 n'est ni édité ni remplacé par cet écran. L'interface ASCRC est une console indépendante de gestion opérationnelle des candidats et des ressources. Elle n'a aucun bouton « déclarer vrai/faux » : le statut épistémique est affiché en lecture seule depuis son référentiel autorisé. Toute correction épistémique doit passer par le mécanisme compétent, avec provenance et trace séparées.

13.2 Vue ASCRC

Le tableau de bord contient : plafond de ressources, ressources consommées et réservées, files par classe de travail, candidats actifs/dormants/bloqués/périmés/révoqués, demandes de réveil, tâches protégées, prochains candidats servis si le plan est déterministe, causes de report, quotas configurés et journal d'arbitrage. Les ressources sont vectorielles au niveau de la spécification (temps, mémoire, processeur/accélérateur, appels, stockage, énergie si instrumentée); le prototype peut présenter un coût scalaire lorsqu'il est le seul coût réellement fourni, avec cette limitation affichée.

13.3 Politique d'allocation

Les contrôles de politique incluent plafond dur; fractions éventuelles exploration/vérification/maintenance; score utilité/nouveauté/information/coût/risque/âge; seuil de vieillissement; fréquence d'activation; limites par plan; règles de protection; conditions de réveil; politiques d'invalidation et reprise de checkpoint. Les budgets réservés et le budget consommé sont visuellement distincts. Une fraction configurée mais non appliquée par le contrôleur doit être indiquée comme non supportée et ne peut pas apparaître comme active.

Toute modification de politique montre un diff, une validation des contraintes, un aperçu simulé, les effets sur les files et un moyen de revenir à la précédente. Le lancement exige de confirmer les hypothèses de ressources. La simulation n'est jamais présentée comme une exécution réelle.

13.4 Veille, réveil, protection et équité

Le candidat ne peut être endormi si une transaction, une lease ou une protection explicite l'interdit, sauf procédure distincte de rupture de protection dûment autorisée. Le réveil déclenché par un événement n'ignore pas le budget ni les contrôles de sécurité. L'écran précise qu'un signal de réveil est une demande, non une garantie d'activation immédiate.

L'interface peut afficher un indicateur de vieillissement et un objectif d'équité, mais elle ne doit pas annoncer une garantie anti-famine sans montrer les hypothèses requises (admissibilité continue, coût payable, opportunités de service, absence de blocage permanent et reprise sans perte). La trace d'arbitrage enregistre candidats examinés, filtres, raisons d'admission/rejet opérationnel, allocation, reports et version de politique. Ces raisons ne changent pas le statut épistémique du candidat.

14 Évaluations, modèles du monde et décisions de promotion

14.1 Centre d'expérimentation

Le centre d'évaluation permet de créer une campagne, figer le protocole, définir métriques et critères pratiques, sélectionner jeux de test ou scénarios, enregistrer seed et version de code, exécuter répétitions, comparer les baselines, analyser les écarts et publier un rapport. Le protocole et les critères doivent être enregistrés avant l'examen des résultats finaux lorsque l'objectif est confirmatoire.

14.2 Évaluation d'un modèle du monde

Le modèle du monde reste optionnel et sa promotion dépend de son bénéfice décisionnel mesuré. L'écran compare au minimum un contrôleur sans modèle et la variante utilisant le modèle, sur des tâches appariées et à budget comparable; il peut ajouter modèles nuls/permutés, ablations, analyses d'erreurs, sécurité et coût. Une précision de prédiction ou une métrique latente favorable n'établit pas seule une amélioration de décision. Les résultats historiques O62/O63 sont étiquetés comme éléments historiques rapportés, non comme une nouvelle exécution. L'interface ne doit pas prétendre qu'une campagne a été relancée si elle ne l'a pas été.

14.3 Rapport de campagne

Le rapport inclut : protocole versionné; données et exclusions; taille d'échantillon; seeds; résultats par cas et agrégats; incertitude; effets pratiques; coûts; violations de sécurité; cas d'échec; limites; statut de reproductibilité; liens vers traces et configurations. Toute métrique dont l'unité, la direction ou la méthode d'agrégation sont inconnues est marquée non comparable plutôt que mélangée à d'autres scores.

15 Objectifs, tâches et travail autonome contrôlé

La vue « Objectifs et tâches » expose les objectifs ouverts, sous-objectifs, critères d'achèvement, dépendances, priorités, budget, échéances si définies, blocages, essais, preuves requises et politique d'arrêt. Les tâches longues conservent leur état et peuvent être reprises lorsque le service prend en charge checkpoint et reprise. L'utilisateur peut inspecter et suspendre les autorisations accordées à une tâche.

Le système n'est pas autorisé à élargir silencieusement son objectif, ses permissions, les données accessibles ou le budget. Une découverte de dépendance ou un besoin de capacité absente crée une proposition de modification accompagnée de son motif. Une tâche partiellement réussie conserve ses sorties, les éléments manquants et un plan de reprise explicite. L'interface ne réinterprète pas automatiquement l'absence de succès comme une impossibilité théorique.

16 Sécurité, permissions et isolation

16.1 Règles de sécurité d'interface

- Permissions selon le principe du moindre privilège : lecture, import, exécution, entraînement, accès aux secrets, accès réseau, écriture, promotion et administration sont séparables.
- Les données importées, documents RAG, réponses de modèle externe et contenus de conversation sont des entrées non fiables. Ils ne peuvent modifier les politiques, exécuter du code ou appeler un outil sans contrôle explicite du runtime.
- Les modèles et connecteurs tiers sont isolés dans des processus/conteneurs à permissions minimales lorsque la technologie le permet; taille, temps, réseau et accès fichiers sont plafonnés.
- Les secrets sont gérés par un coffre ou mécanisme équivalent; les journaux affichent des références et masquent les valeurs sensibles.

- Les actions à impact (écriture externe, suppression, déploiement, publication, coût élevé, accès à données sensibles) nécessitent une permission, une prévisualisation et, selon la politique, une confirmation.
- Les événements de sécurité, refus, dépassements de ressources, tentatives de sortie du périmètre et modifications de permission sont audités.

16.2 États et erreurs

Une erreur doit contenir un identifiant de corrélation, l'étape, l'objet et la version concernés, une catégorie, un message utilisateur, un détail technique protégé, une possibilité de reprise si connue et un lien vers la trace. Les erreurs prévues incluent : entrée invalide, format non supporté, ressource insuffisante, permission refusée, conflit de version, dépendance périmée, timeout, quota dépassé, service indisponible, checkpoint incompatible, preuve manquante et résultat non concluant. Un bandeau ne dit « terminé » qu'après confirmation du service.

17 Traçabilité, audit et provenance

Toute opération importante produit un événement append-only avec : identifiant; date/heure et fuseau; acteur/service; commande; portée; versions des entrées et outils; permissions; paramètres; budget prévu/réel si mesuré; statut; sorties et empreintes; erreurs; relations de dépendance; lien de corrélation. Les journaux affichés peuvent être filtrés ou regroupés, mais les exports conservent la structure native et les relations nécessaires à la reconstitution.

La provenance des résultats inclut au minimum les datasets, sources externes, modèles, adaptateurs, compositions, politique ASCRC active, paramètres d'exécution et versions de vérificateur. Le registre d'audit doit différencier une provenance déclarée d'une provenance vérifiée. L'export peut produire un paquet reproductible (manifeste, configurations, hashes, sorties autorisées et rapport) sans inclure automatiquement les secrets ni les données sensibles.

18 Paramètres, préférences et administration

Les préférences utilisateur incluent thème clair/sombre/système, langue, fuseau, densité, panneau contextuel, notifications, raccourcis, confidentialité des aperçus et comportement de sauvegarde. Les paramètres de projet couvrent les référentiels, quotas, règles de collaboration, politique d'export, secrets autorisés, connecteurs, environnement d'exécution et jeux de données. Les réglages système sensibles sont limités à des rôles administratifs distincts.

Les profils d'espace de travail peuvent être enregistrés et partagés sans secrets, incluant disposition, filtres, tables visibles et raccourcis non conflictuels. Les préférences d'affichage ne modifient pas les données ou le comportement scientifique des tâches. Les options de suppression de données affichent le périmètre, les dérivés touchés, les sauvegardes concernées et les obligations de rétention avant confirmation.

19 Modèle d'objets d'interface et contrat d'API

19.1 Objets canoniques

L'interface référence des objets persistants plutôt que d'enregistrer des copies concurrentes dans chaque écran. Les catégories canoniques sont groupées ainsi : (i) travail et dialogue : `Project`, `Conversation`, `Task`; (ii) données et observation : `Observation`, `Dataset`, `DatasetSchema`; (iii) modèle et entraînement : `ModelArtifact`, `TrainingConfig`, `TrainingRun`, `Connector`, `RAGCorpus`; (iv) exécution et composition : `Composition`, `Execution`, `CloneJob`; (v) mémoire et ressources : `MemoryObject`, `ASCRCPolicy`, `ActivationPlan`; (vi) contrôle et provenance : `EvaluationCampaign`, `EvidenceRecord`, `AuditEvent`. Chaque référence inclut un ID opaque, une version, une portée, une date, une provenance, des permissions et un statut pertinent.

19.2 Contrat des services

Les échanges doivent être versionnés, typés et contrôlés. Une commande qui modifie l'état reçoit un identifiant d'idempotence ou un mécanisme équivalent, la version attendue des objets édités et les ressources/permissions nécessaires. Le service retourne l'état constaté, les versions nouvelles, les erreurs structurées et un identifiant de corrélation. Les événements temps réel peuvent être transportés par un mécanisme push ou par interrogation, mais la cohérence finale vient du service qui fait autorité.

Les écrans ne doivent pas inventer une réussite sur la base d'un accusé local. Les états asynchrones comprennent au minimum : brouillon, validé, en attente, en cours, pause demandée, en pause, annulation demandée, terminé, échoué, interrompu, périmé et résultat partiel, dans les combinaisons pertinentes au type d'objet. Les transitions autorisées sont définies par chaque service.

19.3 Couche d'intégration et adaptateurs

La couche d'adaptation sépare l'UI des runtimes natifs, magasins de données, moteurs d'entraînement, modèles importés, moteurs documentaires et services externes. Elle expose ses capacités déclarées, versions et limites. Les écrans s'adaptent à ces capacités sans masquer les différences : format de dataset non supporté, modèle non entraînable, pause non disponible, vue média absente, évaluateur indisponible, et ainsi de suite. Les adaptateurs sont testés indépendamment puis dans des scénarios intercomposants.

20 États transversaux et feedback utilisateur

Chaque vue doit couvrir au minimum les états : chargement; vide avec prochaine action utile; données présentes; filtre sans résultat; en cours; succès confirmé; succès partiel; avertissement; erreur récupérable; erreur terminale; permission refusée; hors ligne; conflit de version; résultat périmé; service indisponible. Les écrans vides donnent un exemple ou une action de départ sans créer de données fictives. Les notifications temporaires ne sont pas l'unique moyen de rendre une erreur critique visible.

Les tâches longues présentent une zone de progression persistante et un accès au journal. Les erreurs évitables sont signalées avant une opération coûteuse, tout en proposant une voie experte lorsque le risque est accepté et autorisé. Les avertissements incluent leur contexte, leur gravité et leur remédiation. Une action impossible pour l'adaptateur courant est désactivée avec la raison et, si possible, la condition pour la rendre disponible.

21 Critères d'acceptation de l'interface

Les critères ci-dessous constituent une base d'acceptation; ils doivent être traduits en tests automatisés, tests manuels et tests système. Le fait de les écrire ne signifie pas qu'ils sont déjà satisfaits.

ID	Critère	Preuve attendue
UI-01	Depuis le chat, créer une tâche structurée et retrouver ses objets liés dans les espaces dédiés.	Test end-to-end avec IDs/version stables.
UI-02	Les erreurs, sorties partielles et étapes non exécutées sont distinguées d'un succès.	Tests d'états et d'erreurs simulées.
UI-03	Importer un dataset dans un format explicitement supporté et inspecter le mapping des champs.	Jeu de test multi-format, schéma et rapport.
UI-04	Détecter/présenter les collisions de clés et risques de fuite entre partitions.	Cas de test dédiés et diagnostic traçable.
UI-05	Configurer, démarrer, suivre, arrêter/reprendre si supporté et comparer un entraînement.	Tests avec backend d'entraînement de référence.

UI-06	Importer un modèle en quarantaine; vérifier provenance, compatibilité et permission avant chargement.	Tests sécurité/adaptateur et refus attendus.
UI-07	Connecter un LLM/RAG optionnel puis désactiver le connecteur sans casser le chemin natif.	Tests avec connecteur simulé et mode sans connecteur.
UI-08	Chaque citation RAG ouvre la source/version correspondante.	Test de provenance et d'invalidation.
UI-09	Observer une session, inspecter l'original et distinguer observation et hypothèse.	Tests de provenance et de statut.
UI-10	Cloner une cible en gardant les liens vers sources et différences; rapporter les cas non testés.	Campagne de clonage reproductible.
UI-11	Éditer une composition et afficher types, effets, contrats, erreurs et disponibilité des opérateurs.	Tests sur opérateurs intégrés et non intégrés.
UI-12	Veiller/réveiller un candidat sans changer son statut épistémique.	Test d'invariant épistémique.
UI-13	Le plan ASCRC respecte les budgets durs et expose les reports et causes.	Tests budgets, plans et traces.
UI-14	Une action bloquée par une protection ou une version périmée est expliquée et n'est pas appliquée silencieusement.	Tests de conflit et atomicité des commandes.
UI-15	Les opérations importantes sont visibles dans l'audit et exportables sans secrets.	Vérification de traces et test anti-fuite.
UI-16	Les flux principaux sont utilisables au clavier et testés avec lecteur d'écran.	Audit WCAG 2.2 AA, automatique et manuel.
UI-17	Les vues restent utilisables sur petit écran sans perdre l'accès aux fonctions essentielles.	Tests responsive sur plusieurs largeurs.
UI-18	Le statut d'implémentation et la disponibilité réelle des adaptateurs sont visibles.	Tests de compatibilité et revue des écrans.
UI-19	Toute promotion de modèle ou composition montre la politique et les tests applicables.	Tests d'autorisation et de promotion.
UI-20	Le produit peut être utilisé sans LLM externe; aucun appel tiers implicite n'est effectué.	Test réseau/permissions en mode natif.

22 Plan d'implémentation conseillé

La réalisation se fait par incréments intégrables, avec des contrats stabilisés tôt et des tests d'interface de bout en bout. Les étapes ne réduisent pas le périmètre final; elles ordonnent la livraison en gardant les fonctions non encore raccordées visibles.

Incrément	**Contenu**	**Sortie de validation**
I0 — Fondation	Système visuel, navigation, gestion de projet, objets canonique, permissions, audit minimal, système d'états et design responsive.	Shell UI + contrats API.
I1 — Chat traçable	Conversations, uploads/références, tâches, résultats structurés, liens aux objets, traces et erreurs.	Parcours chat end-to-end.
I2 — Données	Import, lecteurs, schéma, champs clés, qualité, partitions et provenance.	Tests par format/adaptateur.

I3 — Modèles et runs	Catalogue, chargement sûr, configuration d'entraînement, jobs, checkpoints, métriques et promotion.	Backend de référence + test.
I4 — Composition et mémoire	IDE compositionnel, catalogue de capacités, contrats, exécution sandbox, versionnement, mémoire vivante et restauration.	Matrice opérateur/runtime.
I5 — Observation et clonage	Sessions, timeline, vues médias disponibles, parcours de clonage et mesure d'écarts.	Jeu d'observation reproductible.
I6 — ASCRC	Tableau de bord, budget vectoriel, politique, files, veille/réveil, protection et audit d'allocation.	Tests d'invariants et de budget.
I7 — LLM/RAG	Connecteurs optionnels, coffre de secrets, ingestion documentaire, index, citations et évaluations.	Tests mode natif, sécurité et RAG.
I8 — Consolidation	Campagnes de validation globale, audit d'accessibilité, performances, résilience, test de charge et documentation opérationnelle.	Rapport intégré; limites explicites.

23 Risques d'ergonomie et mesures préventives

Risque	**Symptôme possible**	**Mesure de conception**
Trop d'options visibles	Écran intimidant, actions concurrentes	Progression guidée, sections avancées repliables, vues d'expert.
Chat boîte noire	Difficulté à vérifier les sorties	Liens vers objets, traces, statuts et sources au niveau de chaque résultat.
Fausse maîtrise des données	Mapping incorrect ou fuite de cible	Aperçu, validation explicite des champs clés, partition verrouillée, diagnostics de fuite.
Illusion de compatibilité	Modèle importé mais inutilisable	Capacités/adaptateurs déclarés, test de compatibilité avant chargement.
Illusion de réussite d'entraînement	Perte faible confondue avec efficacité	Séparer train/validation/test, baseline, seuils, rapport de promotion.
Confusion LLM/RAG et vérité	Texte fluent perçu comme preuve	Provenance visible, statut épistémique inchangé, tests et sources inspectables.
Confusion veille/suppression	Objet perdu après allocation	Actions séparées, restauration, checkpoint et simulation d'impact.
Complexité de la composition	Graphe difficile à lire	Minimaps, regroupements, alignement, vue AST, diagnostic local et références stables.
Surcharge des tableaux de bord	Trop d'indicateurs peu actionnables	Affichage hiérarchisé, filtres mémorisés, détail à la demande.
Mauvaise surprise de coût	Opération longue ou coûteuse inattendue	Budget préalable, estimations qualifiées, quotas durs et confirmation contextuelle.

24 Conclusion et statut de la livraison

Cette spécification décrit une interface unifiée dont le chat est l'entrée naturelle, mais dont les objets, workflows et espaces d'expertise permettent de gouverner le système complet : modèles et données, entraînement, connecteurs optionnels, RAG, observation, clonage, programmation compositionnelle, mémoire vivante, allocation ASCRC, évaluation, audit, sécurité et administration. La fluidité résulte de parcours guidés, de références stables, de panneaux contextuels, d'une hiérarchie visuelle cohérente, d'erreurs actionnables, d'états honnêtes et d'une continuité entre conversation et objets persistants.

Le présent fichier est une formalisation de conception. Il ne constitue pas une preuve d'existence de l'application, d'intégration des services ni de satisfaction des critères d'acceptation. Le développement doit relier chaque contrôle à un contrat d'API et à un service réel, puis valider les parcours en environnement intégré. Les fonctions non disponibles restent documentées, visibles et traçables jusqu'à leur intégration; elles ne sont ni effacées du périmètre ni déclarées impossibles par anticipation.

A Matrice de couverture fonctionnelle de l'interface

Domaine FulleGenBrain_4	**Espace UI**	**Objets de continuité**
Dialogue, objectifs et contrôle	Chat / Tâches	Conversation, Task, résultats et trace
Observation multi-vues et DDD	Observation	Observation, session, empreintes, annotations
MG / MGR / MGR2 et relations	Composition / Graphe	Relations typées, dépendances, diff, provenance
ASCR d'eGenBrain v4	Référentiel / vue autorisée	Objet ou service d'origine, lecture sans réécriture
ASCRC	ASCRC	CandidateDescriptor, ActivationPlan, politiques et événements
Programmation compositionnelle	Composition	AST, composant, contrat, effet, exécution et preuve
Synthèse et évolution	Composition / Évaluations	Candidat, contre-exemple, run, promotion et rollback
Clonage / reproduction	Clonage	Cible, sources, clone, critères de fidélité, écarts
Mémoire vivante / vagues	Mémoire	Payload, checkpoint, événement, dépendance, restauration
Modèles et modèle du monde	Modèles / Évaluations	Artefact, version, métriques, décision de promotion
Modèles externes / RAG	Intégrations / RAG	Connecteur, corpus, index, source et citation
Ressources / résilience	ASCRC / Tâches / Traces	Budget, checkpoint, erreurs, reprise et état
Vérification / preuves / audit	Composition / Audit	Obligation, test, certificat, provenance et journal
Sécurité / permissions	Paramètres / Audit	Rôle, permission, consentement, décision et événement

B Règle finale de non-confusion

Pour toute fonctionnalité visible, la fiche d'implémentation doit relier cinq choses : **intention UX**, **contrat d'objet**, **API/service**, **tests de comportement**, **preuve de statut réel**. Un bouton dessiné n'est pas une capacité; une réponse de service n'est pas une preuve de validité métier; un test unitaire n'est pas une validation globale; un composant théorique n'est pas automatiquement exécutable; une réussite partielle doit rester visible. Cette règle est transversale à toutes les surfaces de l'interface.